

学生成绩数据库系统期末项目报告

数据库及实现 | 第 19 组

项目	内容
系统名称	学生成绩数据库系统
小组成员	章赵岳、胡海漫、陈瑞轩
技术栈	Python Flask、MySQL、PyMySQL、HTML/CSS/JavaScript、Jinja2
GitHub 仓库	https://github.com/zhangzhaoyue2006/student-grade-database-system

一、系统需求分析

1.1 项目背景与目标

本项目面向高校教学管理场景，设计并实现一个用于管理学生成绩的数据库系统。系统围绕学生、院系、专业、课程、教师、授课安排、选课记录和成绩记录等数据展开，目标是建立规范、低冗余、易维护的关系型数据库，并提供基于 Web 的交互式管理界面。我们的系统建设重点包括以下几个方面：

- 建立满足第三范式的数据库模型，减少冗余和更新异常。
- 实现基础信息、授课安排、选课记录、成绩录入、成绩查询和统计分析等核心业务。
- 实现管理员、教师、学生三类用户的权限隔离，保护学生个人信息和成绩数据。
- 提供完整的使用说明和项目文档，覆盖系统安装部署、启动运行、测试账号和功能验证流程，便于最终验收。

1.2 用户角色与权限需求

角色	主要职责	权限边界
管理员	维护院系、专业、课程、教师、学生、授课、选	可查询成绩和统计结果，但不录入、

	课、用户和日志等系统数据。	修改或删除成绩。
教师	查看本人授课课程和学生名单，对本人授课课程进行成绩管理和统计查看。	只能操作本人绑定课程；已结课课程只能查询，不能再修改成绩。
学生	查看个人信息、按学期查询选课和成绩。	只能查看本人数据，不能访问他人信息和管理功能。

1.3 核心业务流程

- 基础信息维护：管理员登录后进入对应模块，对院系、专业、课程、教师、学生等基础数据进行新增、查询、修改和删除。
- 授课安排：管理员选择教师、课程和学期，设置授课角色与课程状态，形成教师与课程之间的多对多关系。
- 选课管理：管理员为学生添加选课记录，系统通过学生、课程、学期联合主键避免重复选课。
- 成绩管理：教师进入本人授课课程，选择未结课课程录入或修改成绩；课程结课后只允许查询。
- 成绩统计：管理员或教师选择课程后查看完成人数、实际选课人数、平均分、中位数、最高分、最低分、及格率、优秀率和分数段分布。

二、数据库概念模型设计

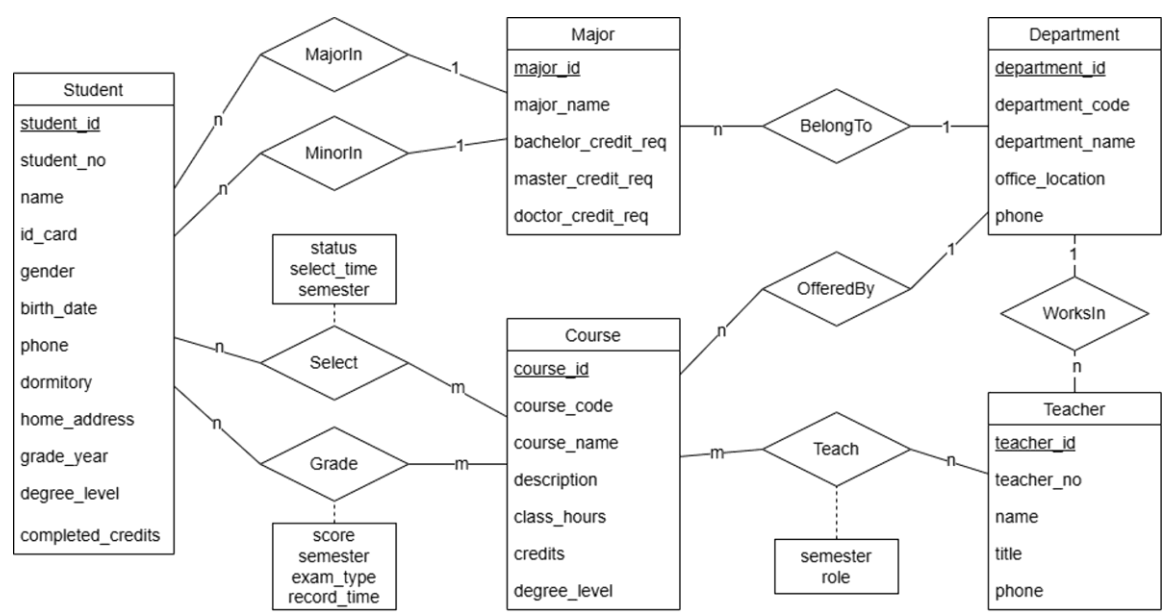


图 1 学生成绩数据库系统 ER 图

2.1 实体划分

系统在期中设计的 6 个核心实体基础上，结合最终实现增加了用户、系统日志和职称等辅助实体，从而支持权限管理、操作审计和教师职称维护。核心业务仍围绕学生成绩管理展开。

实体	含义	关键属性
Department	院系	院系代码、院系名称、办公地点、电话
Major	专业	专业名称、所属院系、本科/硕士/博士学分要求
Student	学生	学号、姓名、身份证号、性别、年级、主修专业、辅修专业、学位等级
Course	课程	课程编号、课程名称、课程说明、学时、学分、学位等级、开课院系
Teacher	教师	教师编号、姓名、所属院系、职称、电话
Title	职称	职称名称、职称等级
Teacher_Course	授课联系	教师、课程、学期、授课角色、课程状态
Student_Course	选课联系	学生、课程、学期、选课状态、选课时间
Grade	成绩	学生、课程、学期、分数、考试类型、记录时间
Users	用户账号	用户名、密码哈希、角色、绑定学生/教师、启用状态、是否强制改密
System_Log	系统日志	操作用户、操作类型、详情、时间

2.2 实体联系关系

关系	类型	实现方式
Department - Major	1:n	major.department_id 外键关联 department
Major - Student	1:n	student.major_id 外键关联 major; student.minor_id 表示辅修专业

Department - Course	1:n	course.department_id 外键关联 department
Department - Teacher	1:n	teacher.department_id 外键关联 department
Title - Teacher	1:n	teacher.title_id 外键关联 title
Teacher - Course	n:m	通过 teacher_course 联系表实现, 复合主键为 teacher_id、course_id、semester
Student - Course	n:m	通过 student_course 联系表实现, 复合主键为 student_id、course_id、semester
Student/Course - Grade	1:n	grade 同时外键关联 student 与 course, 并记录成绩属性
Users - Student/Teacher	0/1:1	学生账号绑定 related_student_id, 教师账号绑定 related_teacher_id

三、数据库逻辑设计

3.1 关系模式与主外键设计

数据表	主键 / 唯一约束	主要外键与业务约束
department	department_id; department_code、 department_name 唯一	院系代码和名称非空唯一
major	major_id; UNIQUE(department_id, major_name)	department_id → department; 学分要求 ≥ 0
student	student_id; student_no、 id_card 唯一	major_id/minor_id → major; 性别、学位等级、年级范围检查
course	course_id; course_code 唯一	department_id → department; 学时和学分大于 0
title	title_id; title_name 唯一	title_level > 0
teacher	teacher_id; teacher_no 唯一	department_id → department; title_id → title

teacher_course	(teacher_id, course_id, semester)	teacher_id → teacher; course_id → course; 授课角色和课程状态检查
student_course	(student_id, course_id, semester)	student_id → student; course_id → course; 选课状态检查
grade	grade_id; UNIQUE(student_id, course_id, semester, exam_type)	student_id → student; course_id → course; 分数 0-100
users	user_id; username 唯一	绑定 student/teacher; 角色、启用状态、强制改密状态检查
system_log	log_id	user_id → users, 可在用户删除后置空

3.2 第三范式分析

- 第一范式：各表字段均为不可再分的原子值，例如学生姓名、学号、课程编号、成绩分数均独立存储。
- 第二范式：复合主键表中的非主属性依赖完整复合主键，例如 student_course 的选课状态依赖学生、课程和学期整体。
- 第三范式：非主属性之间不存在传递依赖，例如学生表只保存 major_id，不重复保存院系名称；教师表只保存 title_id，不重复保存职称名称和等级。
- 多对多关系拆分为独立联系表，避免在实体表中重复存储多个课程或多个教师信息。

3.3 完整性约束与业务规则

约束类型	设计内容	作用
实体完整性	各表设置主键，自增 ID 或复合主键保证记录唯一。	防止表中出现无法唯一识别的记录。
唯一约束	学号、身份证号、课程编号、教师编号、院系代码、用户名等设置 UNIQUE。	防止关键业务编号重复。
参照完整性	通过外键维护学生-专业、课程-院系、教师-院系、成绩-学生/课程等关系。	防止孤立记录和非法引用。
检查约束	分数 0-100，学时/学分大于 0，角色和状态字段限	防止无效数据进入数据库。

	制取值范围。	
业务规则	课程已结课后成绩不能再修改；管理员不能修改成绩；学生只能查看本人数据。	保障教学管理流程和成绩数据安全。

四、功能设计与模块划分

系统功能按照登录角色划分为管理员端、教师端和学生端。管理员端负责全局数据维护和系统管理，教师端负责本人授课课程的成绩与统计，学生端负责个人选课和成绩查询。因此，本节按照管理员端、教师端和学生端三个角色分别梳理功能模块，明确各角色的功能入口、主要操作和对应实现，如下表所示。

4.1 管理员端功能模块

功能模块	主要功能	对应实现
管理员首页	展示管理员可进入的管理入口，作为后台功能导航。	routes/dashboard.py, templates/dashboard.html
基础信息管理	维护院系、专业、课程、教师、职称和学生基础数据，支持新增、查询、编辑和删除。	departments、majors、courses、teachers、titles、students 路由
授课管理	维护教师、课程和学期之间的授课安排，设置授课角色和课程已结课/未结课状态。	routes/teaching.py, teacher_course 表
选课管理	为学生添加或删除选课记录；添加表示选课，删除表示退课或取消选课。	routes/enrollments.py, student_course 表
成绩查询	管理员可查询成绩记录和学生成绩情况，但不能录入、修改或删除成绩。	routes/grades.py, grade 表
统计分析	先按开课学期或院系筛选课程，再进入单门课程查看完成人数、平均分、最高/最低分和分数段分布。	routes/statistics.py
用户账号管理	新增账号、绑定角色、启用/停用账号、重置密码，并限制停用当前管理员或最后一个启用管理员。	routes/system.py, users 表
系统日志	查询登录、增删改、启停账号、重置密码等操作记录。	routes/system.py, system_log 表

4.2 教师端功能模块

功能模块	主要功能	对应实现
教师首页	展示教师端功能入口，进入授课课程、学生名单、成绩管理和统计页面。	routes/dashboard.py
授课课程与学生名单	按学期筛选本人授课课程，进入课程详情后查看当前选课学生名单及所属院系。	routes/teacher_portal.py, teacher_course/student_course 表
成绩录入与查询	未结课课程可进入成绩管理页录入或修改成绩；已结课课程只能查询，不能继续修改。	routes/grades.py, grade 表
课程成绩统计	先选择学期，再选择本人该学期课程，查看平均分、中位数、最高/最低分、及格率、优秀率和分数段分布。	routes/statistics.py
修改密码	教师可在登录后修改本人密码；管理员重置后首次登录需先修改密码。	routes/auth.py, users 表

4.3 学生端功能模块

功能模块	主要功能	对应实现
学生首页	展示学生端功能入口，进入个人信息、选课查询和成绩查询页面。	routes/dashboard.py
个人信息查看	查看本人学号、姓名、专业、院系、年级、学位等级等基础信息。	routes/student_portal.py, student/major/departement 表
我的选课	先选择学期，再查看本人该学期有效选课课程，并显示该学期选课学分和累计已修学分。	routes/student_portal.py, student_course/course 表
我的成绩	按学期查看已结课课程的最终有效成绩；已结课但无成绩显示为缺考，补考或重修取最高分作为最终有效成绩。	routes/grades.py, grade 表
修改密码	学生可在登录后修改本人密码；管理员重置后首次登录需先修改密码。	routes/auth.py, users 表

五、系统实现

5.1 系统架构

系统采用 Flask + MySQL 的典型 Web 应用架构。前端页面由 Jinja2 模板渲染，用户请求进入 Flask 路由层，路由层通过 PyMySQL 连接 MySQL 数据库完成数据读写。页面样式集中在 static/css/style.css 中维护，数据库连接配置通过 .env 和 config.py 管理。

层次	实现内容
表现层	HTML 模板、CSS 样式、少量 JavaScript；提供登录页、列表页、表单页和详情页。
业务层	Flask Blueprint 按模块划分路由，包含权限校验、表单校验、业务规则判断。
数据访问层	models/db.py 提供 get_connection()，各路由通过 PyMySQL 参数化查询访问数据库。
数据层	MySQL 数据库，schema.sql 定义表结构、约束和外键，seed.sql 提供测试数据。

5.2 后端实现要点

- 模块化路由：每类业务建立独立 Blueprint，例如 courses、students、grades、statistics、system 等，便于维护。
- 权限控制：通过 login_required 和 role_required 装饰器限制未登录用户和越权访问。
- 成绩权限：管理员不允许修改成绩；教师只能操作本人授课课程，且课程状态为已结课时只能查询。
- 密码安全：用户密码不明文存储，使用 Werkzeug 的 password hash 保存 password_hash。
- 防 SQL 注入：数据库操作使用 PyMySQL 参数化查询，用户输入通过 %s 占位符绑定。
- 日志记录：对新增、修改、删除、重置密码、启停账号等 POST 操作写入 system_log。

5.3 前端实现要点

- 统一布局：base.html 提供顶部导航、消息提示和页面容器，减少重复代码。

- 列表-详情结构：学生名单、教师统计和管理员统计均采用先筛选列表，再进入独立详情页的交互方式。
- 表单页面设计：新增和编辑页面统一设置保存与返回按钮，保证各模块操作方式一致，降低用户误操作概率。
- 状态区分：账号启用/停用、课程已结课/未结课、缺考等状态使用文字或颜色提示。
- 统计可视化：成绩统计详情页以指标卡片和分数段分布条形图展示统计结果。

5.4 运行与部署说明

本节只展示数据库系统在本地环境中的基本启动流程，完整安装步骤、环境配置和测试账号说明详见项目 README.md。

请按照顺序执行以下操作：

1. 进入项目根目录，例如：cd student-grade-database-system。
2. 根据 env.example 创建 .env 文件，并填写 DB_PASSWORD 等数据库连接配置。
3. 确认 MySQL 服务已启动后，执行 database\init_db.bat 初始化数据库和测试数据。
4. 执行 .\venv\Scripts\python.exe run_dev.py 启动 Flask Web 系统。
5. 在浏览器访问 http://127.0.0.1:5001 进入系统登录页面。

系统初始化完成后，可使用以下测试账号分别登录管理员端、教师端和学生端，用于验证不同角色的功能入口和权限控制。

角色	测试用户名	测试密码
管理员	admin	admin123
教师	teacher01	teacher123
学生	student01	student123

六、测试与功能验证

测试项	测试内容	预期结果
登录测试	分别使用管理员、教师、学生账号登录。	进入对应角色首页，未登录访问会跳转登录页。
权限测试	学生账号登录后手动输入管理员功能路径、管理员尝试进入成绩修改、教师访问非本人课程。	系统拒绝越权操作并给出提示，学生界面不会显示管理员功能入口。
基础数据测试	新增、查询、编辑、删除院系/专业/课程/教师/学生。	数据正确写入，唯一约束和外键约束正常生效。
授课与选课测试	新增授课安排、设置课程状态、添加学生选课记录。	关联表数据正确生成，重复数据被复合主键限制。
成绩测试	教师为未结课课程录入和修改成绩，再将课程设为已结课。	未结课可维护成绩，已结课只能查询。
统计测试	管理员和教师进入课程统计详情页。	显示完成人数、平均分、最高/最低分、及格率、优秀率和分数段分布。
用户管理测试	新增用户、重置密码、启用/停用账号。	初始密码后首次登录需改密，停用账号无法登录。

七、GitHub 仓库地址

项目代码已上传至 GitHub 公共仓库：

<https://github.com/zhangzhaoyue2006/student-grade-database-system>

八、项目总结

本项目从高校学生成绩管理需求出发，完成了数据库概念模型设计、逻辑表结构设计、完整性约束设计和 Web 系统实现。数据库通过主键、外键、唯一约束和检查约束保证数据一致性，并通过联系表处理教师-课程、学生-课程等多对多关系。系统功能覆盖管理员、教师、学生三类用户的主要业务场景，能够完成基础数据维护、授课安排、选课管理、成绩管理、统计分析和用户系统管理。

从实现角度看，系统采用 Flask Blueprint 进行模块拆分，使用 MySQL 存储核心业务数据，使用参数化 SQL 提升安全性，使用密码哈希和角色权限控制保护账号与成绩数据。整体上，系统基本满足课程项目对数据库规范性、功能完整性、界面实用性和文档完备性的要求。